

MPI

MPI

MPI, *Message Passing Interface*, is a *standard* for writing parallel programs on **distributed** memory systems.

MPI is a *library* that provides functions for `C` and `Fortran` (and some other languages) to send *messages* between processes.

As well as some *global* messages between all processes called **collective** communications

We'll go over MPI, its functions, and some issues present when using a memory passing model

And finally revisit performance

Installation

MPI is available on most systems and it's primarily used for high-performance computing clusters.

We can install it on our local machines where each core is treated as a separate process and acts like a node in a cluster.

[mpich.org](https://www.mpich.org) is a popular implementation of MPI.

For windows, it leads you to `MS-MPI` which is a Microsoft implementation of MPI.

For apple, you can install `mpich` using `brew install mpich` .or `sudo port install mpich`

This should mean you have

```
1  mpicc
2  mpirun
```

Available on your command line.

Hello world

Here is a simple "Hello World" program in MPI:

```
1  #include <stdio.h>
2  #include <string.h>
3  #include <mpi.h>
4
5  const int MAX_STRING = 100;
6
7  int main(void) {
8      char greeting[MAX_STRING];
9      int comm_size;
10     int my_rank;
11
12     MPI_Init(NULL, NULL);
13     MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
14     MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
15     // ...
```

Hello world

```
1    // ...
2    if (my_rank  $\neq$  0) {
3        sprintf(greeting, "from process %d of %d", my_rank, comm_size);
4        MPI_Send(greeting, strlen(greeting) + 1, MPI_CHAR, 0, 0, MPI_COMM_WORLD);
5    } else {
6        printf("from processes %d of %d %s\n", my_rank, comm_size);
7        for (int q = 1; q < comm_size; q++) {
8            MPI_Recv(greeting, MAX_STRING, MPI_CHAR, q, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
9            printf("%s\n", greeting);
10        }
11    }
12 }
```

Compilation and execution

```
1  mpicc -Wall -o mpi_hello mpi_hello.c
```

`mpicc` is a *wrapper* for the `gcc` compiler that *links* the MPI library.

And you can also use `mpirun` or `mpiexec` to run the program with a specified number of processes:

```
1  mpirun -n 4 ./mpi_hello
```

Setup

```
1  #include <mpi.h>
```

Imports the MPI library and allows you to use MPI functions

Initialization

```
1 MPI_Init(NULL, NULL);
```

Determine which processes are *participating* in the MPI program.

No MPI functions should be called *before* the `MPI_Init`.

The specific syntax for `MPI_Init` can vary, but the most common form is

```
1 MPI_Init(int* argc_p, char*** argv_p);
```

Where `argc_p` and `argv_p` are pointers to the arguments passed in main. When a program has no arguments, we can simply pass `NULL` for both parameters.

Note that `MPI_Init` returns an *int* error code, we'll ignore it

Finalization

```
1  MPI_Finalize();
```

This states that we're done using MPI, and that *any resources allocated by MPI can be freed*.

The syntax is similar, but it takes no arguments and returns an error code:

```
1  int MPI_Finalize(void);
```

Communicators

In MPI, a *communicator* is a **collection of processes** that can send messages to each other

`MPI_Init` defines a communicator that consists of *all the processes started by the user* when they started the program

This is usually called `MPI_COMM_WORLD`, and we can get information out of this communicator using the following functions:

```
1 MPI_Comm_size(MPI_Comm comm, int* comm_size_p);  
2 MPI_Comm_rank(MPI_Comm comm, int* my_rank_p);
```

The first argument is the *communicator*, with a type defined by `MPI`,

And these functions *return to a location*, usually to an *address* defined in the *second argument*

SPMD

Notice that this is **one** program that is run by *multiple* processes.

Most MPI programs are written this way, and this is called **SPMD** (Single Program, Multiple Data).

We defined our spmd primarily using *if else* statements, but we could also use *switch* statements or even *function pointers* to define the behavior of each process.

Notice that our program can, in principle, run with any number of processes which is good practice

And while that property isn't required, it is generally a good idea as it allows for better scalability

Communication

Communication

In our program, other than process 0, each process *creates a message* which will *send* to 0

```
1     if (my_rank  $\neq$  0) {
2         sprintf(greeting, "from process %d of %d", my_rank, comm_size);
3         MPI_Send(greeting, strlen(greeting) + 1, MPI_CHAR, 0, 0, MPI_COMM_WORLD);
4     } else {
5         // ...
```

Note that we're using `sprintf` which is a *string* version of `printf`

And process 0 simply prints its own message, then *receives* messages from all other processes.

```
1     printf("from processes %d of %d %s\n", my_rank, comm_size);
2     for (int q = 1; q < comm_size; q++) {
3         MPI_Recv(greeting, MAX_STRING, MPI_CHAR, q, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
4         printf("%s\n", greeting);
```

MPI_Send

```
1  int MPI_Send(  
2      void* msg_buf_p,  
3      int msg_size,  
4      MPI_Datatype msg_type,  
5      int dest,  
6      int tag,  
7      MPI_Comm communicator  
8  )
```

The first three arguments, `msg_buf_p`, `msg_size`, and `msg_type` specify the message to be sent.

1. `msg_buf_p` is a pointer to the *data to be sent*,
2. `msg_size` and `msg_type` go hand-in-hand to specify the *type of the data* and *how much of that type* to be sent.

MPI_Send

The next three arguments, `dest`, `tag`, and `communicator` specify *where the message should be sent*.

1. `dest` is the *rank* of the process that should **receive** the message,
2. `tag` is an integer that can be used to **label** the message, and
3. `communicator` specifies the **communicator** that the sender and receiver belong to.

You can think of `tag` as a way to *filter* messages.

An `communicator` as a *world*, and you can have multiple communicators that represent different groups of processes. And they would have *no way* of communicating with each other.

Tags

Suppose that `process 0` receives strings, and depending on some computation, it will either *print* or *store* the string.

The first four arguments of `MPI_Send` provide no information on what `process 0` should do

But we can use a `tag`, for example, `process 1` can use tags `0` `1` `2` etc. to define behavior

- `process 2` uses tag `1` because it wants to print
- `process 3` uses tag `2` because it wants to store
- `process 1` uses tag `0` because it wants to do something else

Communicator

However, say that we're studying global climate change, and we have *two modules*, one for `atmosphere modeling` and one for `ocean modeling`. And both of these modules use *MPI*

Since they were written independently, they **can't** communicate with *each other*, but they can communicate *internally*

Our job would be to write *interface* code for the two processes.

And while we could come up with a scheme for tags, like `ocean modeling` uses tags `0-99` and `atmosphere modeling` uses tags `100-199`, this is *error prone* and *not scalable*.

Remember that a *communicator* is essentially a **separate** universe, and processes in one communicator **cannot** communicate with processes in another communicator.

MPI_Recv

```
1  int MPI_Recv(  
2      void* msg_buf_p,  
3      int msg_size,  
4      MPI_Datatype msg_type,  
5      int source,  
6      int tag,  
7      MPI_Comm communicator,  
8      MPI_Status* status  
9  )
```

The first three, similarly to `Send` ,

- `msg_buf_p` ,
- `msg_size` , and
- `msg_type` specify where the received message should be stored.

`msg_buf_p` *points* to the location to save it, and `msg_size` and `msg_type` specify the *type* and *amount* of data to be received.

MPI_Recv

- the `source` argument specifies the *rank of the process that sent the message*

In *MPI* the receiver usually has to know *who* sent a message, with exception

- the `tag` argument specifies the tag of the message to be received
- the `communicator` argument specifies the communicator that the sender and receiver belong to

Which should **equal** to the `communicator` and `tag` used in `MPI_Send` for the message to be received

- the `status` argument is a pointer to an `MPI_Status` *structure* that can be used to get information about the received message, such as its size and source.

Most of the time, we can *ignore* the `status` argument by passing `MPI_STATUS_IGNORE` .

Message matching

Suppose process 1 uses MPI_Send with

```
1 MPI_Send(send_buf_p, msg_size, msg_type, dest, send_tag, communicator);
```

Then process 0 can receive this message using MPI_Recv with

```
1 MPI_Recv(recv_buf_p, msg_size, msg_type, source, recv_tag, communicator, &status);
```

The message sent by 1 will **only be received** by 0 if the following conditions are met:

1. `recv_communicator = send_communicator`
2. `recv_tag = send_tag`
3. `source = 1`
4. `dest = 0`

MPI_ANY_SOURCE

It's likely that a process is receiving messages from *multiple* sources, and it may not know *which* source will send a message first.

If processes `1, 2, ...` send their results back to `process 0` when they finish

Because the time it takes to finish the work is *unpredictable*, `process 0` has no way of knowing the order in which the results will be sent back.

What we *could* do, and what we did do for the example, is to *place* the sent messages into a buffer and wait for a loop to receive them.

MPI_ANY_SOURCE

But this means that,

for example, if process 2 *finishes first*, process 0 will still *wait* for process 1 to finish before it can receive the message from process 2 .

Standard practice is to use MPI_ANY_SOURCE and MPI_ANY_TAG to receive messages as they come in, *without waiting for a specific source or tag*.

And handle the order of messages after

MPI `status_p` argument

The receiver, especially when using `MPI_ANY_SOURCE` and `MPI_ANY_TAG`, could receive a message without knowing

1. the *sender of the message*, or
2. the *tag of the message*

This information can be obtained from the `status_p` argument of `MPI_Recv`, which is a pointer to an `MPI_Status` struct. Which is the last argument of `MPI_Recv` and is used to get information about the received message.

`MPI_Status` is a struct with *at least* 3 members

- `MPI_SOURCE` which gives the rank of the sender of the message
- `MPI_TAG` which gives the tag of the message
- `MPI_ERROR` which gives the error code of the message

MPI status

Suppose that our program defines a variable

```
1 MPI_Status status;
```

and after a call to `MPI_Recv` where `&status` is passed as the last argument, we can use

```
1 status.MPI_SOURCE  
2 status.MPI_TAG
```

To get the source and tag of the received message.

And even

```
1 MPI_Get_count(&status, msg_type, &count);
```

Which will return the number of elements received

Send and Recv Summary

```
1  int MPI_Send(  
2      void* msg_buf_p,      // input  
3      int msg_size,         // input  
4      MPI_Datatype msg_type, // input  
5      int dest,             // input  
6      int tag,              // input  
7      MPI_Comm communicator // input  
8  )
```

i.e.

```
1  MPI_Send(  
2      greeting,  
3      strlen(greeting) + 1,  
4      MPI_CHAR,  
5      0,  
6      0,  
7      MPI_COMM_WORLD  
8  );
```

```
1  int MPI_Recv(  
2      void* msg_buf_p,      // output  
3      int msg_size,         // input  
4      MPI_Datatype msg_type, // input  
5      int source,           // input  
6      int tag,              // input  
7      MPI_Comm communicator, // input  
8      MPI_Status* status    // output  
9  )
```

i.e.

```
1  MPI_Recv(  
2      greeting,  
3      MAX_STRING,  
4      MPI_CHAR,  
5      1,  
6      0,  
7      MPI_COMM_WORLD,  
8      MPI_STATUS_IGNORE  
9  );
```

Semantics of Send and Recv

1. When `MPI_send` is called, and moves onto the next line, it *doesn't guarantee* that the message has been transmitted
2. `MPI_Recv` is *blocking* so it will freeze until it receives a message that matches the specified source, tag, and communicator.
3. `MPI_Send` from the **same** process is *non-overtaking*,
 - so if `process 1` sends two messages to `process 0`, the first message sent by `process 1` must be available to `process 0` before the second message.
4. But there is **no** restriction on **arrival time** of messages sent from **different** processes, - for example, if `process 1` is running on a machine on Mars,
 - while `process 2` and `3` are both running on the same machine in San Francisco,
 - and if `process 1` sends its message a nanosecond before `2` sends its message
 - it would be extremely unreasonable to require that `process 1`'s message arrive before `process 2`'s message.

Potential Pitfall

Notice that `MPI_Recv` tries to receive a message that matches the specified source, tag, and communicator.

If it **does not** find one, the program will *hang*

So when designing an MPI program, we need to make sure that for every `MPI_Recv` there is a corresponding `MPI_Send` that matches the source, tag, and communicator.